

A Model-Checking Approach for OSEK/VDX Application Verification

June 21, 2026

Abstract

OSEK/VDX applications consist of multiple tasks whose execution order is determined by the OSEK/VDX scheduler. During execution, tasks invoke standardized application interfaces (APIs), such as `ActivateTask`, `TerminateTask`, `WaitEvent`, and `SetEvent`, to change task states, synchronization events, shared resources, and the ready queue. Therefore, a checking model for OSEK/VDX applications should reflect the behaviours of the scheduler instead of treating these APIs as ordinary function calls only.

Existing model checking methods can exhaustively check concurrent software, but they may become imprecise when the OSEK/VDX scheduler is not represented explicitly. If unnecessary task interleavings are explored during verification, the checker may report spurious bugs. This increases the effort required from developers, because they must determine whether a reported bug is a real error or only an artifact of an inaccurate checking model.

This report presents a checking-model construction approach for OSEK/VDX applications. The approach has two main parts. First, it represents the effects of OSEK/VDX APIs on task states, events, shared resources, the ready queue, and task execution order. Second, it uses a manifest to record selected program locations where interrupt service routine (ISR) calls should be inserted. The final checking model contains guarded ISR calls only at these selected locations, rather than inserting ISR calls broadly across the program.

The experimental results show that the proposed approach improves the verification process for OSEK/VDX applications. It builds a more accurate checking model, reduces unnecessary interleavings, removes spurious bugs reported by the baseline checking process, and makes the checking process clearer.

1 Introduction

OSEK/VDX [1, 2] is an automotive operating-system standard introduced in 1994 by European automobile manufacturers, suppliers, and the University of Karlsruhe. It was created to manage the increasing amount of software in vehicles while improving product quality and reliability. The standard has been adopted by major car manufacturers such as BMW, Opel, and Toyota, and is used to support vehicle applications such as cruise control and temperature control. Because these applications directly affect vehicle operation and driver safety, ensuring their reliability is essential. However, as automotive software becomes more complex, verifying the correctness and reliability of OSEK/VDX applications remains a significant challenge.

1.1 OSEK/VDX Applications and Model Checking

An application developed on OSEK/VDX OS consists of multiple tasks. To concurrently execute multiple tasks on one CPU, OSEK/VDX scheduler uses *static priority scheduling policy* with non-preemptive strategy and full-preemptive strategy to determine the running task. In particular, Figure 1 shows that a ready queue in scheduler is used to lay out the execution order of tasks. OSEK/VDX OS can process two types of tasks, basic task and extended task. The states of a

basic task consist of *running state*, *ready state* and *suspended state*. Compared with the basic task, the extended task can hold synchronization events and has a unique state called *waiting state*.

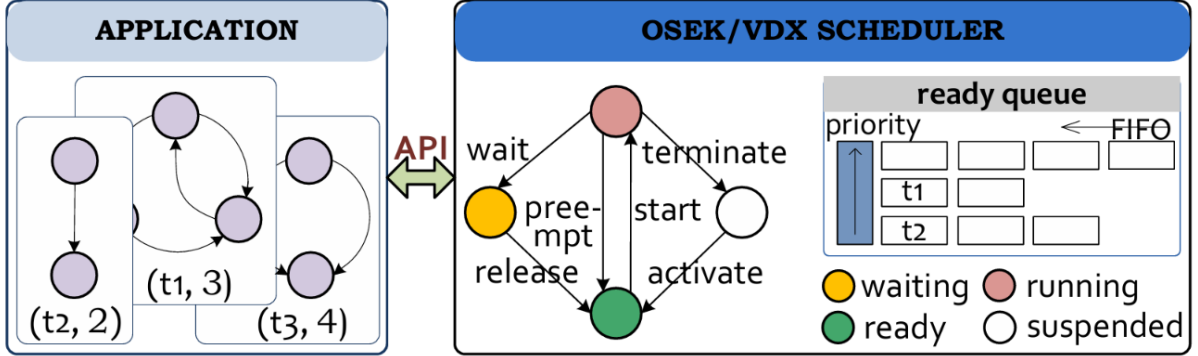


Figure 1: OSEK/VDX scheduler

OSEK/VDX OS also provides standardized application interfaces (APIs), such as **ActivateTask**, **TerminateTask**, **SetEvent**, and **WaitEvent**, through which tasks can dynamically change task states, synchronization events, and shared resources.

To ensure the reliability of OSEK/VDX applications, testing is an important checking technique used in industry, but it is difficult to exhaustively check OSEK/VDX applications using testing technique, especially because of incomplete test-cases. In addition, it is hard to reproduce an error trace when an error is detected by testing, due to the concurrency of OSEK/VDX applications.

Model checking [3, 4], an efficient and exhaustive verification technique for concurrent software, can be applied to verify OSEK/VDX applications instead of testing technique. Particularly, it will return a counterexample when detecting an error. Based on the returned counterexample, developers can easily find the error trace and reproduce the erroneous behaviours.

1.2 Shortcomings of Existing Model Checking for OSEK/VDX Applications

Existing model checking methods [5, 6, 7], such as the ANSI-C multi-threaded software and SystemC multi-threaded software, have been applied to verify concurrent software. In these methods, the behaviours of scheduler are taken into account in the verification process. However, these prominent model checking methods are still unsuitable to accurately verify OSEK/VDX applications because of different scheduling policy. As shown in Figure 2, in the existing methods, the running thread within the concurrent software is arbitrarily determined by scheduler (non-deterministic scheduler is used to dispatch threads), but in OSEK/VDX applications which task is to be run is explicitly determined by OSEK/VDX scheduler, a deterministic scheduler used to dispatch tasks.

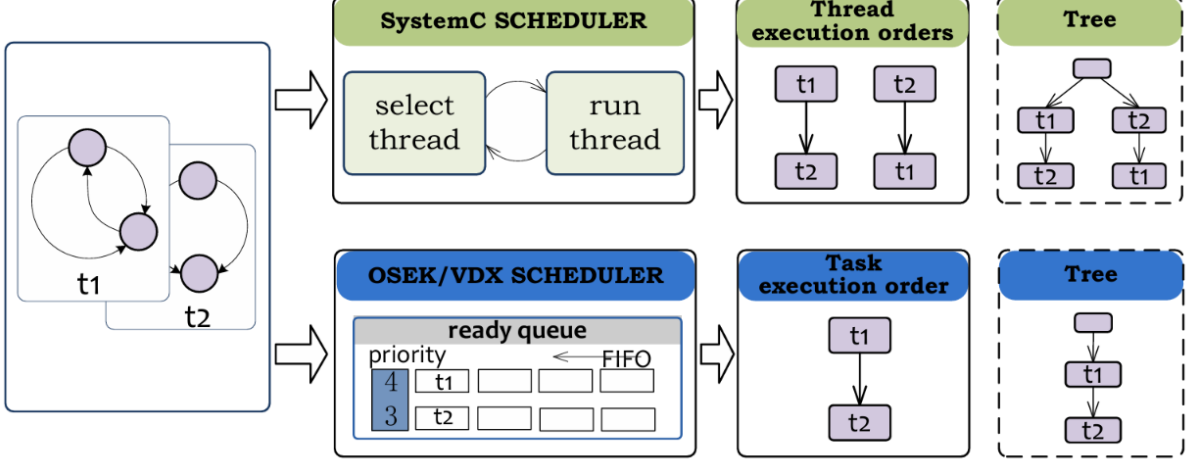


Figure 2: The difference of schedulers

If such methods are applied directly to OSEK/VDX applications, the verification process becomes too imprecise because the behaviours of OSEK/VDX scheduler are not taken into account. As a result, a large number of unnecessary interleavings of tasks may be checked in the verification stage, which can lead to spurious bugs and increases the checking costs in terms of judging time and memory of checking devices.

1.3 Thesis Purpose & Contribution

The purpose of this report is to make model checking more accurate for OSEK/VDX applications. In order to achieve this purpose, we describe and develop an approach to construct a checking model that reflects their execution semantics and avoid checking ISR occurrences at program locations that do not affect the checking results.

In our approach, we introduce how the checking model is constructed before the final verification stage. During checking-model construction, a recognized OSEK/VDX API call is not treated only as an ordinary C function call. Instead, it updates the representation of the current execution state, including the running task, the ready queue, the waiting tasks, and the stored event state of each relevant task. After processing a recognized API call, the checking model updates the current scheduling state. From this updated state, the deterministic scheduler determines what is checked next: execution may remain in the current task, switch immediately to a ready task, return to the interrupted context that preceded the current task, or finish the current task and start another ready task. In addition, some recognized API calls write the stored event state back to program variables, so later branch conditions and assertions can depend on that state. As a result, the checking model is constructed from these state updates and the resulting dispatch decisions, rather than from the ordinary local control flow of the current task alone.

To avoid checking unnecessary interleavings, our approach also constructs a manifest, which is an intermediate description used in checking-model construction: for each ISR, the manifest records the shared or global state that the function can write, and the source locations where that function needs to be represented in the checking model. This manifest is built by identifying the ISR or interleaving functions under consideration, summarizing the shared or global state that each of them can modify, and then scanning the remaining program to find the program points where reads, writes, branch conditions, or scheduling effects can be changed by that state. The final checking model then introduces guarded ISR behaviour only at the source locations recorded in the manifest, so that the verification still reflects the task execution order determined

by the deterministic scheduler while avoiding unnecessary interleavings and reducing spurious bugs.

The report makes three contributions:

- It describes a checking approach for OSEK/VDX applications whose task execution order is determined by a deterministic scheduler, task states, synchronization events, and a ready queue.
- It introduces a manifest-guided method for selecting ISR insertion locations and building a checking model with ISR calls only at the program locations recorded in the manifest.
- It presents benchmark results showing that the proposed approach reduces unnecessary interleavings and removes spurious bugs reported by the baseline checking process.

2 Background

2.1 OSEK/VDX OS

A general OSEK/VDX OS consists of a scheduler module, synchronization event process module, shared resource process module, alarm process module and interruption process module. Based on these system modules, OSEK/VDX OS supports a standardized application interfaces (APIs) for user to develop customized applications. In our research, we focus on the applications that communicate with scheduler module, synchronization event process module and resource process module. The structure of OSEK/VDX OS with an application is shown in Figure 3.

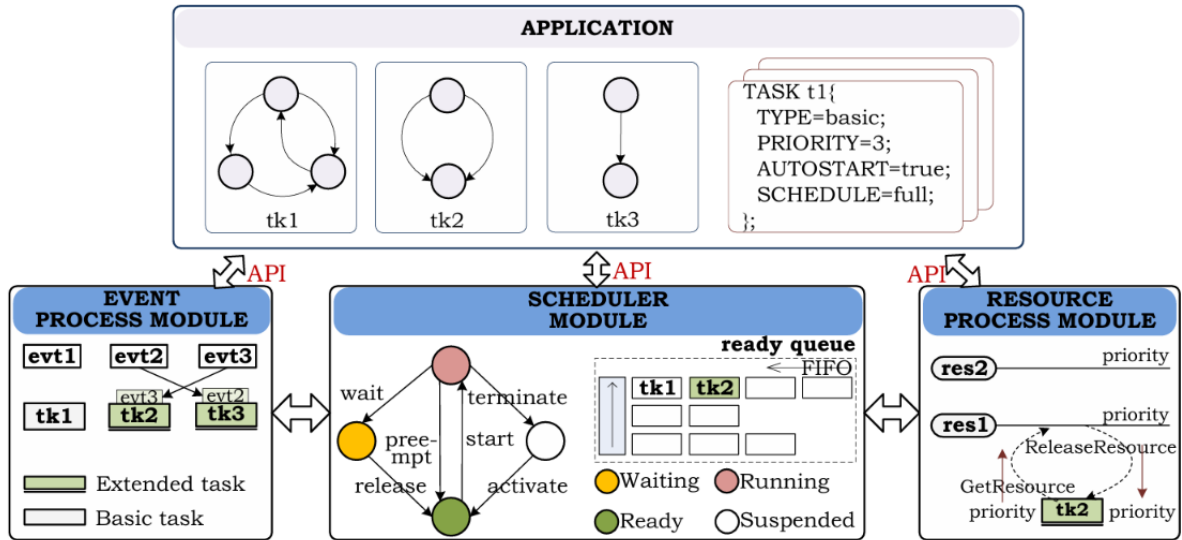


Figure 3: The structure of OSEK/VDX OS with an application

2.1.1 Scheduler Module

OSEK/VDX OS can process two types of tasks, basic task and extended task. The states of a basic task consist of *running state*, *suspended state*, and *ready state*. Compared with the basic task, the extended task can hold synchronization events and has a unique state called *waiting state*. In the scheduling process, the *static priority scheduling policy* with non-preemptive and full-preemptive strategies is adopted by the scheduler to conduct the executions of tasks. Particularly, in OSEK/VDX OS, scheduler manages a ready queue to lay out the execution order

of tasks. Based on the ready queue, the running task is explicitly determined by OSEK/VDX scheduler.

In addition, scheduler module provides several APIs for applications, such as *TerminateTask*, *ActivateTask* and *ChainTask*. Tasks within the application can invoke these APIs to dynamically change the states of tasks, and the changed states will affect the scheduling of tasks, e.g., the context switch of tasks will happen when a suspended task is activated by running task using API *ActivateTask* or *ChainTask*. For instance, as shown in Figure 4, there are two tasks *t1* and *t2*, and the priority of task *t1* is lesser than *t2*. Currently, task *t1* is running task. When the API *ActivateTask(tk2)* is invoked by *t1*, scheduler will move task *t2* from suspended state to ready state. At this moment, the context switch of tasks will happen, because the priority of task *t2* is higher than *t1*.

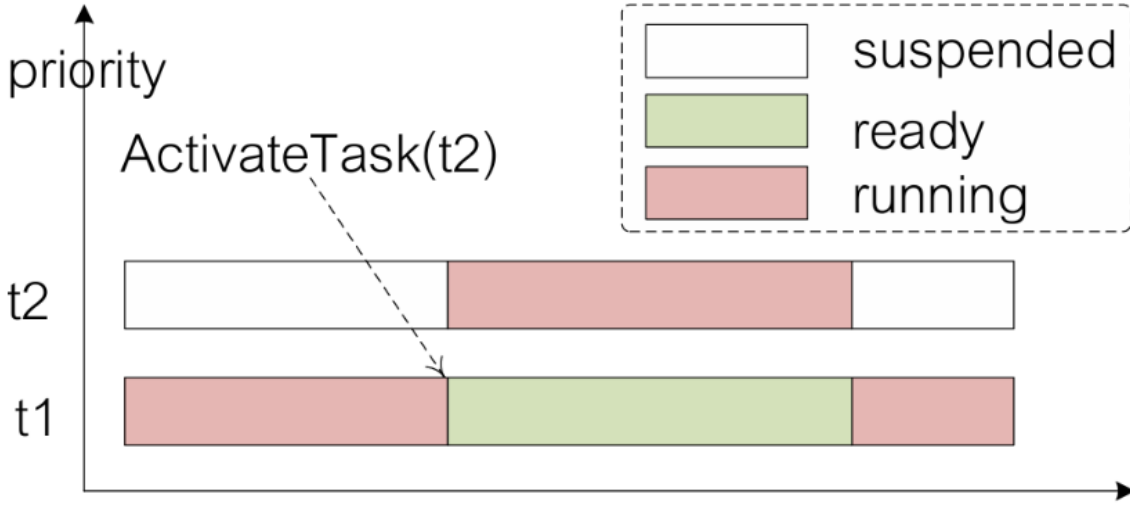


Figure 4: The context switch of tasks and API *ActivateTask*

2.1.2 Event Process Module

In the synchronization event process module, OSEK/VDX OS provides a synchronization mechanism for implementing synchronous executions between tasks. Particularly, only extended tasks can hold a definite number of events, and events are the criteria for the switching of task states from *running state* to *waiting state* or from *waiting state* to *ready state*. There are three APIs *SetEvent*, *WaitEvent* and *ClearEvent* that can be responded by event process module, and tasks within application can invoke these APIs to implement the synchronous behaviours. E.g., as shown in Figure 5, when the running task *t1* waits for the event *evt1* using API *WaitEvent(evt1)*, task *t1* cannot continue until the event *evt1* is set by other tasks (basic tasks or extended tasks) using API *SetEvent(t1,evt1)*.

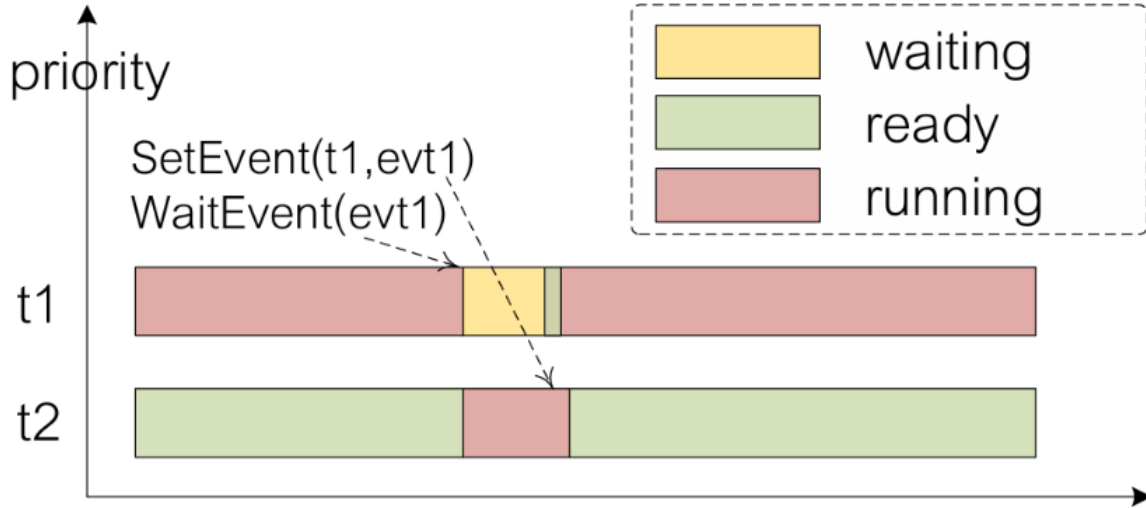


Figure 5: The synchronous behaviours between tasks

2.1.3 Resource Process Module

OSEK/VDX OS adopts the *Priority Ceiling Protocol* [8] to coordinate the behaviors of task accessing shared resources in the resource process module. The resource process module supports two APIs (*GetResource* and *ReleaseResource*) that can be invoked by tasks to access shared resources. E.g., as shown in Figure 6, if the API *GetResource(res1)* is invoked by running task *t1*, and the priority of the task is lower than the ceiling priority of the resource *res1*, the priority of the task will be raised to the ceiling priority of the resource *res1*, and the priority of the task will be reset to the priority before requiring the resource *res1* when *ReleaseResource(res1)* is invoked by the task. Note that, the ceiling priority of a shared resource is lower than the lowest priority of all tasks that do not access the resource, and it is higher than the priorities of all tasks that access the resource.

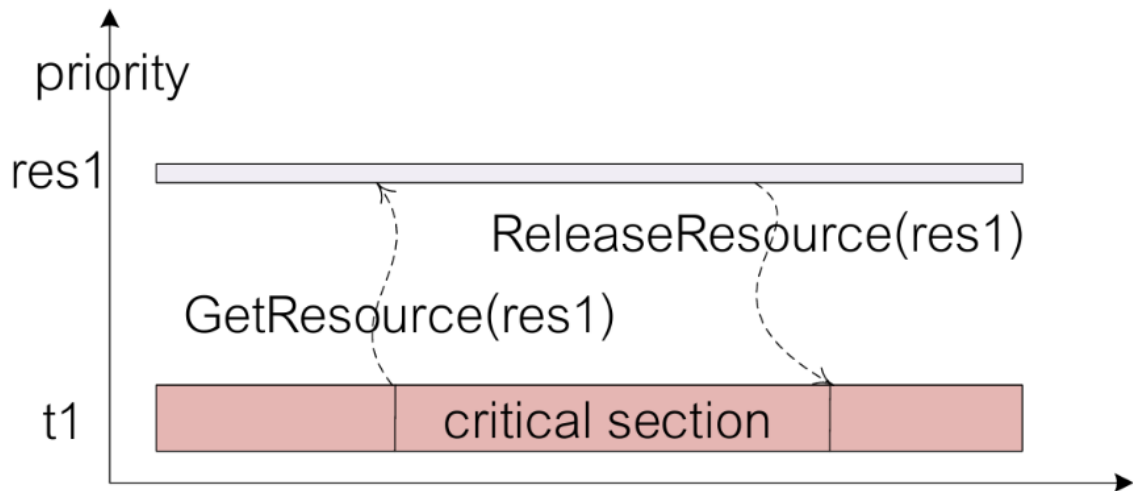


Figure 6: The priority ceiling protocol

2.2 OSEK/VDX Application and Execution Characteristics

2.2.1 OSEK/VDX Application

An application developed based on OSEK/VDX OS consists of two files, one is the source file, and the other is the configuration file. The source file which can be developed by C language is used to present the concrete behaviors of the application. The configuration file is used to define tasks, synchronization events and shared resources. In the configuration file, the attribute **AUTOSTART** is used to set the initial state of tasks. If the attribute **AUTOSTART** of a task is set to be **TRUE**, the task starts from ready state in the initial state (it will be inserted into ready queue according to the priority of the task). Otherwise, the task starts from suspended state. The attribute **SCHEDULE** is used to indicate the scheduling type. If the attribute **SCHEDULE** of a task is set to be **FULL**, the task can be preempted by higher priority tasks. Otherwise, the task cannot be preempted by higher priority tasks. A simple OSEK/VDX application without synchronization events and shared resources is shown in Figure 7. As to clearly comprehend the execution characteristics of OSEK/VDX applications, an example is symbolically executed in this part. In the application shown in Figure 7, only the attribute **AUTOSTART** of **t1** is set to be **TRUE**. Thus, **t1** will be firstly moved to running state by scheduler and then executed in the initial state.

Source file (.cpp)	Configuration file (.oil)
<pre>int a, b, c, h; TASK t1(){ if(b >= a){ a++; ActivateTask(t2); } else{ b++; ActivateTask(t3); } ActivateTask(t4); TerminateTask(); } TASK t2(){ if(h > 0) h--; else h++; TerminateTask(); } TASK t3(){ c=b; c=(c+b)/b; TerminateTask(); } TASK t4(){ assert(a > b); TerminateTask(); }</pre>	<pre>TASK t1{ TYPE=BASIC; SCHEDULE=FULL; PRIORITY=5; AUTOSTART=TRUE; }; TASK t2 { TYPE=BASIC; SCHEDULE=NON; PRIORITY=6; AUTOSTART=FALSE; }; TASK t3 { TYPE=BASIC; SCHEDULE=NON; PRIORITY=4; AUTOSTART=FALSE; }; TASK t4 { TYPE=BASIC; SCHEDULE=NON; PRIORITY=4; AUTOSTART=FALSE; };</pre>

Figure 7: The structure of OSEK/VDX OS with an application

As shown in Figure 8, when the API *ActivateTask(t2)* is invoked by *t1* (the branch“(b >= a)” is chosen to symbolically execute), scheduler will be loaded to respond to the API (task *t2* is moved from suspended state to ready state). At this moment, the running task *t1* will be preempted by *t2* since the priority of *t2* is higher than *t1* and the attribute **SCHEDULE** of *t1* is set to be **FULL** (the context switch happens after the API *ActivateTask(t2)*). Currently, task *t2* gets run-unit to run, and goes to suspended state when the API *TerminateTask()* is invoked (*TerminateTask()* is used to terminate the executions of a task, and the terminated task will be

moved from running state to suspended state by scheduler. If the running task is terminated, scheduler then dispatches the head task in the ready queue to run when ready queue is not empty). When task t_2 terminates itself using API *TerminateTask()*, scheduler will dispatch task t_1 to running state. Then, t_1 continues its executions from the preempted point, and task t_4 is activated by t_1 using API *ActivateTask(t_4)* (the context switch will not happen, since the priority of task t_4 is less than t_1). Finally, task t_4 will be executed when the running task t_1 terminates itself using API *TerminateTask()*. Thus, according to the conducted symbolic executions, we can get a task execution sequence t_1, t_2, t_1, t_4 in the execution path π_1 and π_2 . However, if the API *ActivateTask(t_3)* is invoked by task t_1 (the “branch $!(b \geq a)$ ” is chosen to symbolically execute), we will get a different task execution sequence t_1, t_3, t_4 in the execution path π_3 .

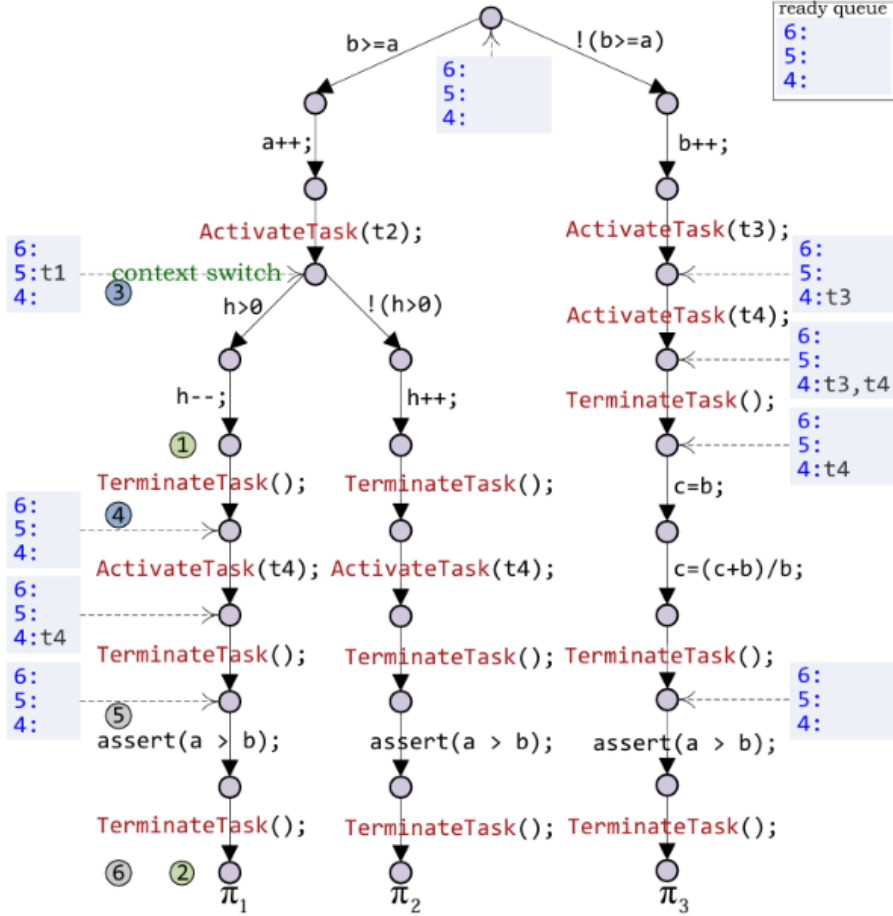


Figure 8: The execution tree for the application shown in Figure 7

2.2.2 Execution Characteristics of OSEK/VDX Application

Based on the execution tree of the application shown in Figure 7, we can easily find the following execution characteristics of OSEK/VDX applications:

- Which task is to be run is explicitly determined by the OSEK/VDX scheduler according to the ready queue and the configuration data of the application.
- The APIs invoked from tasks may haphazardly change the scheduling of tasks. For example, different service APIs located at different branches may lead to different task execution sequences, and a task context switch may happen when an API is invoked by the running task.

According to the listed execution characteristics, there are several challenges that should be addressed when we apply model checking techniques to verify an OSEK/VDX application, for example:

- How to precisely construct a model to represent the executions of the application.
- How to deal with the OSEK/VDX OS, for example, how to explicitly perform the scheduling behaviours of the OSEK/VDX OS, and how to handle the APIs invoked from tasks.

2.3 Model Checking

Concurrent software usually consists of multiple threads. To concurrently execute multiple threads on one CPU, a scheduler is used to dispatch threads. There have been many concurrent software standards such as SystemC standard, POSIX standard and ANSI C standard that have been widely used in the industry of software. In these concurrent software standards, which thread within software is to be run is arbitrarily determined by scheduler. In other words, the running thread is stochastic when the multi-threaded software runs on the machine. If testing technique is used to check such software, it is difficult to reproduce an error track because of the non-deterministic scheduling. In contrast to testing technique, model checking can solve the problem based on the reported counterexample.

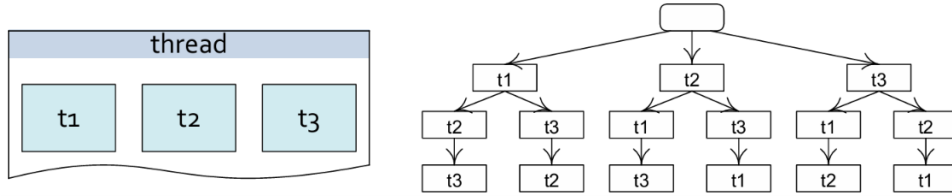


Figure 9: The model for a SystemC multi-threaded software

The key idea of model checking is: firstly construct a model to describe the executions of a target system, and then search the constructed model with given property. To verify concurrent software, a tree as a model is usually constructed for demonstrating all of the possible interleavings of threads. E.g., as shown in Figure 9, there is a SystemC multithreaded software, and the multi-threaded software consists of three threads. In model checking, a tree shown in the right side of Figure 9 as a model will be constructed to exhaustively describe all of the possible executions of the multi-threaded software. When the model of concurrent software is constructed, there existing many checking techniques that can be applied to search the constructed model for checking whether the constructed model satisfies the given property or not, such as explicit state checking technique [9], binary decision diagrams (BDD) technique [10, 11], symbolic checking technique [12] and bounded model checking technique [13].

Currently, based on the model checking technique, there are many model checkers that have been established based on the different model checking techniques, and these established model checkers have been successfully applied to verify multi-threaded software that conforms to different concurrent software standards. E.g., Bell Labs developed a model checker named **spin** [5] based on the explicit state model checking technique. For the ANSI-C standard, a bounded model checking based model checker named **esbmc** [14] has been implemented in the university of Southampton. In addition, in the group of Cimatti, a model checker named **kratos** [15] has been established for SystemC multithreaded software. There are also several famous model checkers, such as symbolic model checker **SMV** [16], bounded model checker **cbmc** [17], etc.

2.4 Bounded Model Checking

The original motivation of bounded model checking was to leverage the success of satisfiability (SAT) [18, 19] in solving Boolean formulas to model checking. Compared to explicit state model checking technique and BDD-based model checking technique [10, 11], it offers the advantage of handling the verification of large state spaces. The main idea of BMC is to avoid the full state space generation and look for witnesses of an existential specification on the suitable subsets of the full model under the given bound. Once a sub-model is selected, the considered sub-model will be translated into propositional formulae. Then, the propositional formulae and the negative form of given specification (given property) will be solved by a specialized SAT solver. If the verification result is true, it means that the sub-model does not satisfy the given specification, a counterexample will be report by the SAT solver. Otherwise, a larger sub-model will be selected and the whole procedure will be run again until the whole model is verified.

3 Methodology

3.1 Overview of the Proposed Approach

The proposed approach focuses on how the checking model is constructed before final verification. It does not depend on replacing a particular verification tool. Instead, the main goal is to build a more accurate checking model for OSEK/VDX applications, where task execution order is determined by the deterministic scheduler, the ready queue, task states, and the APIs invoked from tasks.

The approach has two main parts. The first part is to represent the behaviours of the OSEK/VDX scheduler in the checking model. During verification, the model must capture how OSEK/VDX APIs can change task states, synchronization events, shared resources, and the ready queue. This allows the next running task to be determined according to the OSEK/VDX scheduler rather than ordinary C control flow alone.

The second part is manifest-guided interrupt service routine (ISR) insertion. In this report, an ISR is a function that may interrupt the normal execution of tasks. In practice, an ISR may occur at many points during task execution. However, the checking model does not need to represent every possible ISR occurrence separately. The proposed approach focuses on the program locations where an ISR call can affect the checked execution, such as by changing task states, event state, shared variables, the ready queue, or the checked assertion. Other locations are skipped because they only add unnecessary interleavings and increase the checking cost. To avoid this, the proposed approach first computes a manifest, which records the selected program locations where ISR calls should be inserted. The final checking model is then built with guarded ISR calls only at the locations recorded in the manifest. Table 1 summarizes the difference between verification flow and the proposed approach.

The key point is that the proposed approach does not replace the underlying verification tool. Instead, it changes the checking model given to the final verification step. Rather than checking a model that represents scheduling and ISR occurrences too broadly, the approach constructs a checking model that better reflects the executions of OSEK/VDX applications.

The remainder of this section explains the approach in three steps. Section 3.2 describes how OSEK/VDX scheduler behaviours and APIs invoked from tasks are represented. Section 3.3 explains how the manifest is used to reduce unnecessary ISR insertion points. Section 3.4 explains how the checking model that takes the OSEK/VDX scheduler into account and the manifest-guided insertion step are combined before CBMC performs final verification.

To keep the discussion concrete, the remainder of this section uses one synthetic running example. *Task1* is a higher-priority extended task that waits for an event. *Task2* is a lower-priority task that can run while *Task1* is in waiting state. *ISR1* updates shared state and sets the event awaited by *Task1*. The example is intentionally small, but it is large enough to show the three

Aspect	Verification flow	Proposed Approach
Task execution order	Follows the ordinary C program model unless OSEK/VDX scheduling is encoded separately.	Follows task states, the ready queue, and the deterministic scheduler inside the checking model.
OSEK/VDX APIs	May be treated as ordinary function calls if no OSEK/VDX-specific model is provided.	Are handled as operations that can change task states, synchronization events, shared resources, and the ready queue.
Interrupt behaviour	May represent ISR occurrences broadly at many program locations.	ISR calls are inserted only at selected locations recorded in the manifest.
Interleavings	May include unnecessary interleavings that increase the checking cost and may lead to spurious bugs.	Reduces unnecessary interleavings before the final verification step.
Checking model	Represents the program without explicitly combining OSEK/VDX scheduler behaviours and selected ISR locations.	Represents the OSEK/VDX scheduler, changes caused by APIs invoked from tasks, and selected ISR insertion points before final verification.

Table 1: Conceptual comparison between verification flow and the proposed approach.

main decisions in the methodology: how task execution order is determined, how unnecessary interleavings are reduced, and how the final checking model is built.

3.2 Taking the OSEK/VDX Scheduler into Account

The first methodological step is to represent the behaviour of the OSEK/VDX scheduler inside the checking model. In an OSEK/VDX application, the next running task is not chosen by an unstructured interleaving of threads. It is chosen by a deterministic scheduler that depends on task states, priorities, events, and the ready queue. If the checking model ignores those elements, the resulting task execution order no longer matches the execution order of the application.

Listing 1 shows the running example used throughout this section. The example combines waiting, event signaling, shared-state update, and an local computation so that each later step can be explained with reference to the same program.

Listing 1: Pseudo Code

```

int shared_ready = 0;
int shared_value = 0;

TASK(Task1)
{
    EventMaskType observed = 0;
    WaitEvent(EVENT_SHARED_READY);
    GetEvent(TASK_1_ID, &observed);
    if ((observed & EVENT_SHARED_READY) != 0u && shared_ready) {
        int sample = shared_value;
        assert(sample >= 0);
    }
    ClearEvent(EVENT_SHARED_READY);
    TerminateTask();
}

TASK(Task2)
{

```

```

    int checksum = 0;
    checksum = checksum + 1;
    checksum = checksum * 2;
    TerminateTask();
}

ISR(ISR1)
{
    shared_value = 7;
    shared_ready = 1;
    SetEvent(TASK_1_ID, EVENT_SHARED_READY);
}

```

Table 2 summarizes the intended execution context for this example. *Task1* starts as the running task, *Task2* is already ready with a lower priority, and *ISR1* can make *Task1* ready again by setting the event that *Task1* is waiting for.

Entity	Priority / type	Initial status and role
<i>Task1</i>	Higher-priority extended task	Running at the beginning of the path; waits for <code>EVENT_SHARED_READY</code> ; later reads the event state and the shared value.
<i>Task2</i>	Lower-priority task	Ready while <i>Task1</i> is running; may execute while <i>Task1</i> is in waiting state; performs only local computation in this example.
<i>ISR1</i>	Interrupt service routine	Writes <code>shared_ready</code> and <code>shared_value</code> , then sets the event awaited by <i>Task1</i> .

Table 2: Configuration summary for the running example.

The central point is that the standardized APIs in Listing 1 cannot be treated as ordinary function effects. When *Task1* executes `WaitEvent`, the important effect is not only that a C function is called. The important effect is that *Task1* may move from running state to waiting state. Once this happens, the scheduler may select *Task2* as the next running task because *Task2* is already in ready state.

Later, when *ISR1* executes `SetEvent`, the event state changes and *Task1* may move from waiting state back to ready state. Because *Task1* has higher priority than *Task2*, the deterministic scheduler may dispatch *Task1* before *Task2* continues. Therefore, the checking model must represent changes to task states, events, and the ready queue, not only ordinary control-flow changes inside the current C function.

Table 3 is the output of this first methodological step. It shows how task states, the ready queue, and the event state change when the same source program is checked under the OSEK/VDX model.

Step	Current context	Task states	Ready queue	Event state	Scheduler action
0	Start of path	<i>Task1</i> : running; <i>Task2</i> : ready	<i>Task2</i>	Clear	Continue in <i>Task1</i>
1	<i>Task1</i> executes WaitEvent while the event is clear	<i>Task1</i> : waiting; <i>Task2</i> : ready	<i>Task2</i>	Clear	Dispatch <i>Task2</i>
2	<i>Task2</i> executes local computation	<i>Task1</i> : waiting; <i>Task2</i> : running	Empty	Clear	Continue in <i>Task2</i> unless <i>ISR1</i> occurs
3	<i>ISR1</i> updates shared state and executes SetEvent	<i>Task1</i> : ready; <i>Task2</i> : running before dispatch	<i>Task1</i>	Set	Dispatch <i>Task1</i> because it has higher priority
4	<i>Task1</i> continues, observes the event, and reads the shared value	<i>Task1</i> : running; <i>Task2</i> : ready	<i>Task2</i>	Set, then cleared	Continue in <i>Task1</i>

Table 3: State evolution for the running example.

Figure 10 gives a high-level view of how scheduling information is produced in the checking model. When a task invokes an OSEK/VDX API, the API may change task states or events. These changes update the ready queue, and the deterministic scheduler uses the ready queue to decide which task should execute next. Therefore, the checking model must represent this cycle explicitly instead of treating API calls as ordinary function calls.

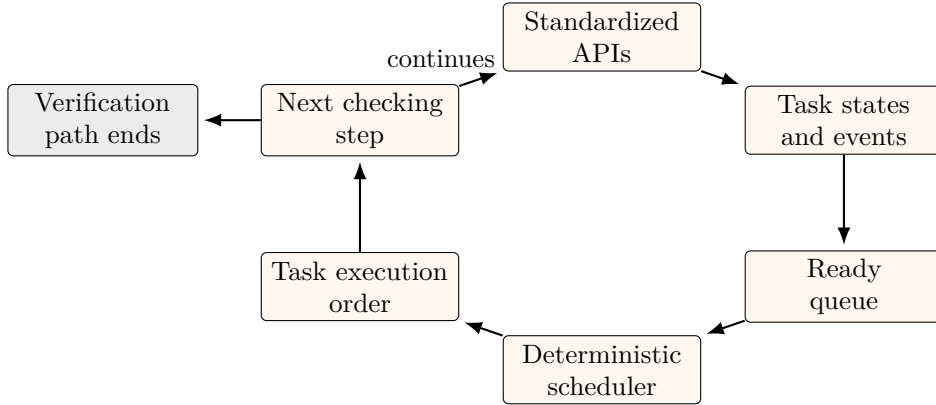


Figure 10: Conceptual cycle from standardized application interfaces (APIs) to task states, the ready queue, the deterministic scheduler, and the next checking step.

Read clockwise, some OSEK/VDX APIs change task states or event state. For example, **WaitEvent** may move an extended task from running state to waiting state when the awaited event is not set, while **SetEvent** may make a waiting task ready again by setting the awaited event. Other APIs, such as **GetEvent**, read event information that is already stored in the checking model. After each relevant API action, the checking model updates the task states, the event state, and the ready queue. The deterministic scheduler then uses this information to determine the next running task. That task execution order determines the next checking step, which may later reach another API call or another state update. If the verification path reaches its end, the loop stops at the termination branch.

In the running example, this means that the verification path should not simply continue

with the next statement in the C code after `WaitEvent` when the awaited event is clear. Instead, the checking model should represent that *Task1* moves to waiting state, that *Task2* may run while *Task1* is waiting, and that *ISR1* can set the event and make *Task1* ready again. In this way, the task execution order in the verification result can reflect the execution order determined by the OSEK/VDX scheduler.

3.3 Reducing Unnecessary Interleavings

After the checking model represents the OSEK/VDX scheduler, the next step is to control where interrupt service routine (ISR) calls are inserted. If ISR calls are inserted at too many program locations, the checking model may include many interleavings that do not affect the property being checked. These unnecessary interleavings can increase the checking costs in terms of states, time, and memory, and may also lead to spurious bugs.

Therefore, the proposed approach first identifies the program locations where ISR calls are relevant to the checked execution. These selected locations are recorded in a manifest. The later rewriting step uses this manifest to insert ISR calls only at those selected locations, instead of adding them broadly across the program.

The key question is therefore not “where could an ISR call be inserted in general?”, The key question is “where can an ISR call change the checking result?”. In the running example, *ISR1* is relevant when its updates to `shared_ready` and `shared_value`, or its call to `SetEvent`, can be observed later by *Task1* or by the scheduler.

If inserting *ISR1* at a program location cannot change a task state, event state, ready queue, or assertion result, then that insertion only adds an unnecessary interleaving to the checking model.

In the running example, *ISR1* has three visible effects: it writes `shared_value`, sets `shared_ready`, and calls `SetEvent(TASK_1_ID, EVENT_SHARED_READY)`. These effects matter when they can be used later by *Task1* or by the scheduler. For example, the call to `SetEvent` can move *Task1* from waiting state back to ready state, and the writes to `shared_ready` and `shared_value` can affect the later `if` statement and assertion in *Task1*.

By contrast, the checksum statements in *Task2* only update the local variable `checksum`. They do not read `shared_ready` or `shared_value`, they do not inspect the event state, and they do not invoke an OSEK/VDX API that changes task states or the ready queue. Therefore, these local statements do not create additional points.

Table 4 shows gives a readable summary of the selected program locations: where an *ISR1* call should be inserted, and where a separate insertion would only add an unnecessary interleaving.

Candidate program location	Can <i>ISR1</i> change results here?	Decision	Reason
Immediately before <code>WaitEvent(EVENT_SHARED_READY)</code>	Yes	Insert	<i>ISR1</i> can set <code>EVENT_SHARED_READY</code> . This can change whether <i>Task1</i> continues or moves to waiting state.
Immediately before <code>GetEvent(TASK_1_ID, &observed)</code>	Yes	Insert	The value stored in <code>observed</code> depends on whether <code>EVENT_SHARED_READY</code> has already been set.
Immediately before the <code>if</code> statement in <i>Task1</i>	Yes	Insert	The branch checks both <code>observed</code> and <code>shared_ready</code> , which can be changed by <i>ISR1</i> .
Immediately before the read of <code>shared_value</code>	Yes	Insert	The value read by <i>Task1</i> is written by <i>ISR1</i> .
Immediately before <code>checksum = checksum + 1</code> in <i>Task2</i>	No	Skip	This statement only updates the local variable <code>checksum</code> . It does not read the event state, <code>shared_ready</code> , or <code>shared_value</code> .
Immediately before <code>checksum = checksum * 2</code> in <i>Task2</i>	No	Skip	This statement is also local to <i>Task2</i> . A separate insertion here would add another interleaving without adding a new observation point in this example.

Table 4: Summary of selected ISR insertion locations in the running example.

The point of Table 4 is not to predict exactly when *ISR1* will occur. The point is to decide where inserting an *ISR1* call is useful for the checking model. The selected locations are the places where *ISR1* can change what *Task1* or the scheduler observes, such as the event state, `shared_ready`, `shared_value`, or the next running task.

The skipped locations are different. In this example, the checksum statements in *Task2* only update the local variable `checksum`. They do not read the event state, they do not read `shared_ready` or `shared_value`, and they do not change task states or the ready queue. Skipping separate ISR insertions at those locations therefore reduces unnecessary interleavings without removing behaviour that can affect the checked path.

3.4 Building the Final Checking Model

The proposed approach combines the previous two steps to build the final checking model. First, it selects the program locations where an *ISR1* call can change what *Task1* or the scheduler observes. These locations are recorded in the manifest. Second, it uses the manifest to build a checking model in which ISR calls appear only at the selected locations.

This separation makes the method easier to control. The selection step answers the question: “where can an ISR call affect the event state, the ready queue, the shared variables, or the assertion checked?” The model-building step answers a different question: how should the final checking model represent only those selected ISR calls?

As a result, the final checking model does not insert ISR calls at every possible program location. It keeps the locations that can affect the checked path and skips locations that only add unnecessary interleavings. This reduces the checking cost while preserving the behaviours needed for the running example.

For the running example, the selected locations can also be summarized in the compact form shown in Listing 2. This summary connects the selection step with the model-building step: it

records where *ISR1* should be inserted, without showing low-level implementation details.

Listing 2: Readable summary of the selected ISR insertion locations for the running example.

Selected ISR insertion locations: 1. Before <code>WaitEvent(EVENT_SHARED_READY)</code> 2. Before <code>GetEvent(TASK_1_ID, &observed)</code> 3. Before the if statement that checks <code>observed</code> and <code>shared_ready</code> 4. Before the read of <code>shared_value</code> Skipped locations: The local checksum updates inside <code>Task2</code>
--

Figure 11 shows how the proposed approach builds the final checking model before bounded model checking. The upper lane is the verification flow. The lower lane adds four stages before the final verification step.

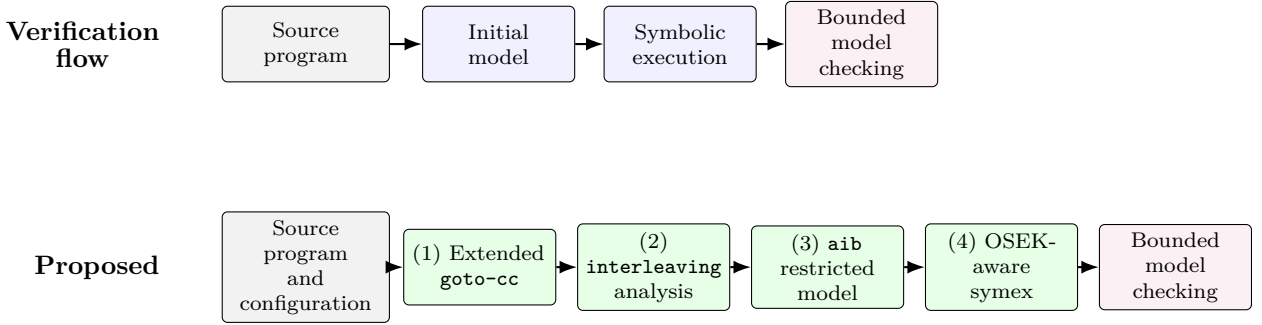


Figure 11: Baseline and improved checking-model construction.

In the verification flow, the source program and the OSEK/VDX configuration are translated into an initial checking model, then we perform symbolic execution and model checking. This path is useful for ordinary C programs, but it does not by itself explain how OSEK/VDX APIs should change task states, events, the ready queue, and task execution order.

The proposed flow adds four stages. First, the extended `goto-cc` stage builds the initial program representation from the source program and the OSEK/VDX configuration. Second, `interleaving-analysis` selects the program locations where ISR calls may affect the checking model. Third, `aib` uses those selected locations to build a restricted checking model, so ISR calls are added only where they are needed. Fourth, `goto-symex` with the `os-api` layer symbolically executes the model while handling OSEK/VDX APIs as scheduler operations rather than ordinary C function calls.

Listing 3 shows the output of the model-building step for the running example. The inserted calls are part of the checking model, not part of the original application program. Each inserted call is guarded by `nondet_bool()`, so model checking may choose whether *ISR1* occurs at that selected location. No ISR call is inserted inside the local checksum updates of *Task2*, because those updates only change the local variable `checksum` and were skipped in Table 4.

Listing 3: Illustrative checking model with selected ISR calls for the running example.

<pre> TASK(Task1) { EventMaskType observed = 0; if (nondet_bool()) ISR1(); WaitEvent(EVENT_SHARED_READY); if (nondet_bool()) ISR1(); </pre>

```

    GetEvent(TASK_1_ID, &observed);

    if (nondet_bool()) ISR1();
    if ((observed & EVENT_SHARED_READY) != 0u && shared_ready) {
        if (nondet_bool()) ISR1();
        int sample = shared_value;
        assert(sample >= 0);
    }

    ClearEvent(EVENT_SHARED_READY);
    TerminateTask();
}

TASK(Task2)
{
    int checksum = 0;
    checksum = checksum + 1;
    checksum = checksum * 2;
    TerminateTask();
}

```

The role of Listing 3 is simple. The final checking model still allows *ISR1* to occur, but only at the selected locations. These locations are kept because *ISR1* can change the event state, the shared variables `shared_ready` and `shared_value`, or the next running task chosen by the scheduler.

By contrast, no ISR call is inserted inside the local checksum updates of *Task2*. Those statements do not read the event state or the shared variables, and they do not change task states or the ready queue. Skipping them reduces unnecessary interleavings while preserving the OSEK/VDX scheduler behaviours needed for the checked execution.

4 Implementation in CBMC and Experimental Evaluation

The previous sections described the checking-model construction methodology in tool-agnostic terms. This section turns to two implementation questions: why CBMC was chosen as the verification host, and how the methodology was realized in that host and evaluated on public interrupt benchmarks.

4.1 Why CBMC

CBMC is a bounded model checker for C programs [13, 17, 20]. It combines front-end translation, symbolic execution, and solver-based checking in one verification flow. That makes it a good implementation target for this work for two reasons. First, the applications studied in this report and the public benchmark families used in the evaluation are written in C. Second, CBMC separates model construction, symbolic execution, and solver back-end checking cleanly enough that the proposed methodology can be inserted without replacing the underlying verification engine. In other words, the present work does not require a new solver. It requires a more accurate checking model before the final solver query is built.

4.2 CBMC Architecture and Realization of the Approach

The baseline CBMC flow used in this report is shown in Figure 12. Source code is translated by the front-end into an internal program representation, that representation is symbolically executed to construct a solver formula, and the solver determines whether the checked property holds under the chosen bound.

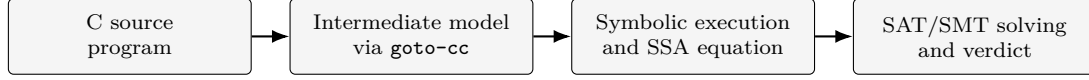


Figure 12: Baseline CBMC architecture used as the implementation host for the proposed approach.

CBMC stage	Baseline role	Added realization	Effect on the checking model
Front-end and goto-cc	Translate C into CBMC's intermediate model.	Extended front-end that reads OSEK/VDX configuration and preserves project context.	Produces the initial checking model used by the later refinement stages.
Separate interleaving analysis	None.	Stand-alone interleaving-analysis pass.	Selects the program points where interruption-written state can affect the checked path.
Restricted-model construction	None.	aib builder on lib-interleaving-block.	Rebuilds the model with guarded interruption behavior only at the selected points.
Symbolic execution and goto-symex	Build the solver formula by symbolic execution.	os-api extension that interprets recognized OSEK/VDX operations during symex.	Symbolic states follow task, event, ready-queue, and dispatch updates rather than local C control flow alone.
Solver back end	Solve the bounded query and reconstruct counterexamples.	Unchanged.	The final decision procedure stays the same; only model construction and execution semantics change before solving.

Table 5: Implementation map from baseline CBMC stages to the realized approach.

This baseline is a strong starting point for C verification, but it is not sufficient by itself for OSEK/VDX applications. The missing part is not symbolic execution. The missing part is the construction of a checking model that reflects scheduler-driven task execution order and that avoids introducing interruption behavior at irrelevant points in the program.

The current branch realizes the methodology inside CBMC by extending the flow in four places. The front-end is extended so that the initial checking model preserves the OSEK/VDX context needed later in the pipeline. A separate analysis stage identifies the program points where interruption-written state can influence the checked execution. A restricted-model builder then reconstructs the checking model with guarded interruption behavior only at those selected points. Finally, symbolic execution is extended so that recognized OSEK/VDX operations update the stored execution view of tasks, events, and dispatch decisions instead of being treated as ordinary C calls.

Figure 13 summarizes the implementation structure. The upper lane is the stock CBMC path. The lower lane shows the added stages: an extended front-end around **goto-cc**, the stand-alone interleaving analysis pass, the restricted-model builder implemented by **aib**, and the OSEK-aware symbolic-execution extension in **goto-symex** through the **os-api** layer. The solver back end is shared. The implementation therefore does not replace CBMC's final decision procedure; it changes how the checking model is built and interpreted before the final query is solved.

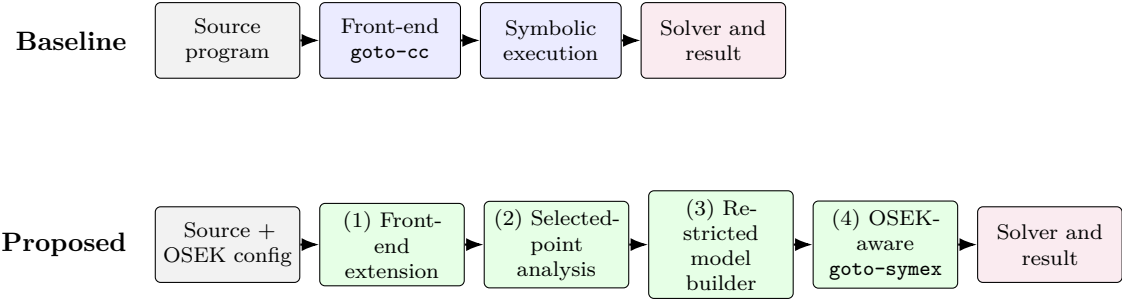


Figure 13: Implementation of the proposed approach in CBMC. The upper lane is the stock flow. The lower lane adds the stages used to construct and execute the restricted checking model.

Family	Origin	Public measured cases	Adaptation for this study
i-CBMC-derived interrupt benchmarks	Public interrupt benchmarks released with i-CBMC.	22	Each case is normalized into a baseline asynchronous model and an improved model that supports selected guarded interruption behavior. The public subset includes larger cases and public diagnostic cases that stress interruption ordering and resource usage.
IntAbs-derived interrupt benchmarks	Public interrupt benchmarks released with IntAbs.	21	Each case is normalized into the same two-model comparison. The improved model keeps the original entry function, checked properties, and bound configuration, while preparing the source so that interruption behavior can be introduced only at selected points.

Table 6: Public benchmark corpus used in the experimental evaluation.

4.3 Benchmark Corpus and Adaptation

The experimental corpus is drawn from two public interrupt-benchmark families: benchmarks derived from i-CBMC [21] and benchmarks derived from IntAbs [22]. Internal smoke cases, simple benchmarks, and Trampoline-based experiments are excluded from the evaluation in this section. After filtering to the public measured corpus, the data set used here contains 43 cases: 22 i-CBMC-derived cases and 21 IntAbs-derived cases.

The adaptation follows the methodology from Section 3. For each public case, a baseline model is preserved in the broad asynchronous style, and an improved model is prepared for restricted checking-model construction. The entry function, checked property setup, and unwind bound remain aligned at the case level. What changes is the way interruption behavior is represented: the improved flow first identifies the interruption entry functions that matter for the case, then rebuilds the checking model so that guarded interruption behavior appears only where it can affect the checked execution.

Outcome class	Cases	Interpretation
Same verification outcome	18	Both flows reach the same verification class for the checked case.
Improved model discharges a baseline spurious alarm	8	The baseline asynchronous model admits an unnecessary interruption ordering, while the improved model removes that spurious behavior and succeeds.
Improved model exposes a behavior missed by the baseline	1	The improved checking model reaches an interrupt-driven error trace that the baseline model proves away.
Unresolved mismatch	7	The two flows disagree, but the current public audit does not yet determine a correct semantic winner.
Not directly comparable	9	The comparison is blocked either by resource limits or by cases where the improved flow has no usable selected interruption points.

Table 7: Outcome classes across the 43 public measured cases.

4.4 Evaluation Questions and Reporting Method

The evaluation addresses three questions. First, does the improved flow discharge spurious alarms caused by unnecessary interleavings in the baseline asynchronous model? Second, does it expose interrupt-driven behaviors that the baseline model fails to represent? Third, what time and memory cost is paid for the added checking-model construction stages?

To answer these questions, each public case is reported in one of five outcome classes. A case has the *same verification outcome* when both flows end in the same verification class. A case *discharges a baseline spurious alarm* when the baseline reports failure but the improved model succeeds and the public audit supports the improved model as the more faithful one. A case *exposes a behavior missed by the baseline* when the baseline succeeds but the improved model reaches an error trace that the public audit supports. An *unresolved mismatch* is a case where the two flows disagree but the current public evidence does not yet justify choosing one model as semantically correct. Finally, a case is *not directly comparable* when the comparison is blocked by a resource limit or by the absence of usable selected interruption points in the improved flow.

All time and memory values in this section are recomputed from the measured public artifacts only. End-to-end time for the baseline flow includes compilation and verification. End-to-end time for the improved flow includes analysis, restricted-model construction, compilation, and verification. Reported per-case totals use median measured phase times, while peak memory is reported from the verification phase.

4.5 Results

4.5.1 Overall outcome summary

Table 7 shows that the improved flow is valuable mainly as a faithfulness refinement, not as a universal acceleration technique. Most importantly, nine public cases are already explained by the public audit as either non-comparable or blocked by pipeline limitations, so they should not be turned into artificial wins or losses. The main positive signal comes from the nine audited cases where the improved checking model changes the verification outcome for semantic reasons

rather than for reporting noise alone.

4.5.2 Representative accuracy gains

The first representative pattern is the removal of a spurious alarm caused by an overly broad asynchronous model. In one logger-oriented case from the i-CBMC-derived family, the baseline flow reports *VERIFICATION FAILED* with a median end-to-end time of 54 984 ms and a median verification-phase peak RSS of 297.4 MB. The improved flow reports *VERIFICATION SUCCESSFUL* for the same bound in 2 025 ms and 16.1 MB. The key point is not the runtime drop by itself. The key point is that the baseline model admits an interruption ordering that is wider than the intended execution model, while the improved flow keeps the interruption entry atomic and therefore removes a spurious bug.

The second representative pattern is the same semantic gain on a device-oriented benchmark with a different cost profile. In one watchdog-style case from the IntAbs-derived family, the baseline flow reports *VERIFICATION FAILED* in 1 202 ms and 13.1 MB, while the improved flow reports *VERIFICATION SUCCESSFUL* in 3 395 ms and 15.3 MB. Here the improved flow is slower end-to-end, but the meaning of the result is stronger: the baseline asynchronous model permits a non-atomic interruption ordering that does not match the intended checked behavior, whereas the improved model narrows the interruption placements and avoids that spurious counterexample.

The third representative pattern goes in the opposite mismatch direction. In another watchdog-style case from the IntAbs-derived family, the baseline flow reports *VERIFICATION SUCCESSFUL* in 1 172 ms and 41.0 MB, but the improved flow reports *VERIFICATION FAILED* in 3 559 ms and 16.7 MB. This case matters because it shows that the improved approach is not only a filter for false alarms. It can also expose an interrupt-driven ordering through shared device state that the baseline model does not represent, leading the improved model to find an error trace that the baseline proves away.

These representative cases should be read together with the unresolved category in Table 7. Seven public cases still disagree without enough public evidence to choose a semantic winner. For a research publication, those cases are limitations to report openly, not hidden wins for either side.

4.5.3 Cost and trade-off

The cost of the improved flow is the extra checking-model construction work performed before final verification. On the 18 public cases where both flows end in the same verification class, the improved flow is slower in 11 cases. Across that same-outcome subset, the median end-to-end time rises from 1 164 ms for the baseline flow to 3 380 ms for the improved flow. This is consistent with the structure of the implementation: even when the final verification phase is not dramatically slower, the added analysis and restricted-model construction stages increase total wall-clock cost.

For the use case of this report, that trade-off is acceptable. The objective is not to make CBMC uniformly faster. The objective is to obtain a checking model that is more faithful to OSEK/VDX execution, reduces unnecessary interleavings, and exposes interrupt-driven behavior more accurately when it matters.

5 Discussion and Limitations

The revised flow is deliberately conservative in some places and intentionally incomplete in others. Those limitations are not incidental; they define where future work should focus.

5.1 What the Current Design Gets Right

The strongest engineering choice in the branch is the separation between the behaviours of OSEK/VDX scheduler and the placement of interruption-related instrumentation. The task execution order is determined during verification, while the candidate program points are determined by an explicit analysis step and recorded in a manifest. This keeps the method maintainable. A change to OSEK/VDX scheduling policy does not automatically require a change to the instrumentation step, and a change to the candidate-line heuristics does not automatically require a change to the verification core.

The evidence in Section 4.5 also suggests that the extra structure is not just architecturally neat; it matters to outcomes. The benchmark report contains multiple cases in which the improved pipeline is audited as the more faithful model even though it is slower.

5.2 Current Analysis Limits

- **Source-file-driven target discovery.** Interleaving functions are selected from an explicit source-file list. If the relevant interruption file is not part of the build, the analysis cannot discover it.
- **Global-write-centric conflict rule.** Candidate lines are derived from globals written by the interleaving function and read or written by the main verification path. This keeps the rule simple, but it ignores purely read-side interactions by the interleaving function.
- **.c-only candidate filtering under project scope.** When project-root filtering is enabled, candidate recording is restricted to .c files under the root. Generated or indirect artifacts outside that slice are ignored.
- **Hand-written manifest IO.** Both the manifest writer and the `aib` reader are hand-coded. There is no schema version field, which makes coordinated changes brittle.

5.3 Current OSEK Model Limits

- **Partial OIL subset.** The parser currently handles the fields needed by the implemented scheduling rules: `TASK`, `PRIORITY`, `SCHEDULE`, `AUTOSTART`, `TYPE`, and `EVENT_MASK`.
- **AUTOSTART is metadata, not boot behavior.** The configuration records `AUTOSTART`, but the current flow still expects explicit task activation from the harness or runtime stubs.
- **WaitEvent is an approximation.** The implementation models the block / no-block decision and wake-up dispatch, but not every possible continuation detail of the full operating-system behaviour.
- **API coverage is intentionally narrow.** The current branch focuses on task and event services relevant to the public benchmarks. Resource, alarm, and interruption-related APIs beyond this subset are not yet modeled through the same boundary.

5.4 Performance Trade-offs

The benchmark report consistently shows that the revised flow pays an overhead tax. Even public cases add a manifest phase, an injection phase, and a rebuild. Larger cases can still be worth that cost when the stock baseline either misses a real bug or invents one through unnecessary interleavings, but the branch is not yet an optimization pass.

This suggests a practical reading of the current status: the branch is already useful as a more accurate bounded model checking workflow, but not yet as a drop-in faster replacement for stock CBMC. Improving analysis precision without paying so much orchestration cost is a natural next step.

6 Conclusion

This report documented a CBMC-based bounded model checking flow for OSEK/VDX applications. The central problem is accuracy: if the deterministic scheduler is ignored, many unnecessary interleavings of tasks may be superfluously checked and may result in a spurious bug. The revised flow addresses that problem by taking the behaviours of OSEK/VDX scheduler into account and by restricting interruption-related instrumentation through an explicit manifest.

The checked-in public benchmark evidence supports that choice. The revised flow is often slower because it adds analysis, instrumentation, and rebuild phases, but it is audited as the more faithful model in meaningful public cases. In particular, it removes unnecessary interleavings that produce spurious bugs and reaches some error traces that are not reached by the stock baseline.

The remaining work is also clear. The supported OSEK/VDX subset can be widened, the manifest contract can be made more robust, and the candidate-line rule can be made less dependent on a simple global-write heuristic. Even in its current form, however, the branch demonstrates a practical path toward more accurate bounded model checking for OSEK/VDX applications.

A Appendix: Interface Examples

A.1 Example goto-cc Invocation

The following command illustrates the public project-mode flow on the `i8xx-tco-1` benchmark tree. It records the project root, names the interruption source file explicitly, and emits an interleaving manifest.

Listing 4: Example project-mode interleaving analysis command.

```
goto-cc \  
  check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/main.c \  
  check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/isr_define/isr.  
  .c \  
  --interleaving-project-root \  
    check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline \  
  --interleaving-source-files \  
    check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/isr_define/  
    isr.c \  
  --interleaving-output \  
    check-src/benchmark-sources/intabs/cases/i8xx-tco-1/interleaving_pipeline.json \  
  -o check-src/benchmark-sources/intabs/cases/i8xx-tco-1/i8xx-tco-1.out
```

A.2 Example aib Invocation

Listing 5: Example AIB rewrite command.

```
aib \  
  check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline \  
  check-src/benchmark-sources/intabs/cases/i8xx-tco-1/interleaving_pipeline.json \  
  out/i8xx-tco-1_injected \  
  out/i8xx-tco-1_injected/interleaving_pipeline_effective.json
```

A.3 Manifest Excerpt

The manifest below is a schematic excerpt showing only the documented public fields. It illustrates how project metadata and interleaving candidates are separated.

Listing 6: Schematic manifest excerpt.

```
{
  "project": {
    "project_root": "/path/to/check-src/benchmark-sources/intabs/cases/i8xx-tco-1/
improved-pipeline",
    "interleaving_source_files": ["isr_define/isr.c"],
    "translation_units": [
      "main.c",
      "isr_define/isr.c"
    ]
  },
  "interleaving": [
    {
      "name": "isr1",
      "write_global_var": ["<global_1>", "<global_2>"],
      "line_added_block_with_file": [
        { "file": "main.c", "line": [<line_a>, <line_b>] }
      ]
    }
  ]
}
```

This schema is the compatibility boundary between `interleaving-analysis` and `aib`; any field change must be implemented on both sides.

References

- [1] J. Lemieux. *Programming in the OSEK/VDX Environment*. Suite 200 Lawrence, KS 66046, USA: CMP, 2001.
- [2] OSEK/VDX Group. *OSEK/VDX Operating System Specification 2.2.3*. URL: <http://portal.osek-vdx.org/>.
- [3] Edmund M. Clarke, E. Allen Emerson, et al. “Model Checking: Algorithmic Verification and Debugging”. In: *Communications of the ACM* 52.11 (2009), pp. 74–84.
- [4] Edmund M. Clarke, Orna Grumberg, and David E. Long. “Model Checking and Abstraction”. In: *ACM Transactions* (1994), pp. 1512–1542.
- [5] Gerard J. Holzmann. *The Spin Model Checker: Primer and Reference Manual*. Boston, USA: Lucent Technologies Inc., Bell Laboratories, Sept. 2003.
- [6] Tony Andrews et al. “Zing: A Model Checker for Concurrent Software”. In: *16th International Conference on Computer Aided Verification (CAV)*. 2004, pp. 484–487.
- [7] B. Wachter, D. Kroening, and J. Ouaknine. “Verifying Multi-Threaded Software with Impact”. In: *Formal Methods in Computer-Aided Design (FMCAD)*. 2013, pp. 210–217.
- [8] Alan Burns and Andy Wellings. *Real-Time Systems and Programming Languages*. 4th. New York, NY, USA: Addison Wesley Longmain, Aug. 2009.
- [9] E. M. Clarke, E. A. Emerson, and A. P. Sistla. “Automatic Verification of Finite-State Concurrent Systems Using Temporal Logic Specifications”. In: *ACM Transactions on Programming Languages and Systems*. 1986, pp. 244–246.
- [10] A. Biere et al. “Symbolic Model Checking without BDDs”. In: *TACAS’99*. Vol. 1579. Lecture Notes in Computer Science. 1999, pp. 193–207.
- [11] E. M. Sentovich. “A Brief Study of BDD Package Performance”. In: *Proceedings of the Formal Methods on Computer-Aided Design*. 1996, pp. 389–403.

- [12] E. Clarke et al. “Symbolic Model Checking”. In: *8th International Conference, CAV ’96*. 1996, pp. 419–422.
- [13] Edmund Clarke et al. “Bounded Model Checking Using Satisfiability Solving”. In: *Formal Methods in System Design* 19 (2001), pp. 7–34.
- [14] Jeremy Morse et al. *ESBMC Homepage*. 2009. URL: <http://www.esbmc.org/>.
- [15] Alessandro Cimatti et al. *Kratos Homepage*. 2010. URL: <https://es-static.fbk.eu/tools/kratos/>.
- [16] A. Cimatti, E. Clarke, and F. “NuSMV: A New Symbolic Model Verifier”. In: *International Conference on Computer-Aided Verification (CAV)*. 1999, pp. 495–499.
- [17] *CBMC*. 2004. URL: <http://www.cprover.org/cbmc/>.
- [18] Mukul R. Prasad, Armin Biere, and Aarti Gupta. “A Survey of Recent Advances in SAT-Based Formal Verification”. In: *International Journal on Software Tools for Technology Transfer* 7 (2005), pp. 156–173.
- [19] Niklas Een et al. “An Extensible SAT-solver”. In: *6th International Conference, SAT 2003*. 2003, pp. 502–518.
- [20] Diffblue Ltd. *CBMC Architecture*. 2026. URL: <https://diffblue.github.io/cbmc/cbmc-architecture.html> (visited on 06/19/2026).
- [21] CProver Project. *Verification of Interrupt-Driven Software*. 2026. URL: <https://www.cprover.org/interrupts/> (visited on 06/19/2026).
- [22] Chungha Sung. *intAbs Repository*. 2026. URL: <https://github.com/ChunghaSung/intAbs> (visited on 06/19/2026).